

Projet SUPFile - stockage cloud distribuée, sécurisée et hautement disponible

Rapport Technique sur le fonctionnement du Frontend :



Datacenter Paris Ayman EL KARROUSSI

Datacenter New York MEVENGUE Franck

Datacenter Toronto Nadia Loukdache

Unité d'enseignement :	Infrastructure & Services Réseau
Promotion :	4 ème année – 2025/2026
Date de Rendu :	Mai 2026

Lien démo App :	https://supfile-webapp.vercel.app/
------------------------	---

Lien repo Github Présentation :	https://github.com/MEVENGUE/SUPFile-VB-App
--	---

Plan Rapport Technique Frontend SUPFile :

1. Introduction :

2. Présentation globale de l'architecture:

- 2.1 Architecture frontend
- 2.2 Infrastructure réseau et communication inter-datacenter :
- 2.3 Contraintes techniques

3. Technologies utilisées :

- 3.1 Frontend
- 3.2 Backend
- 3.3 Infrastructure et sécurité

4. Conception du Frontend :

- 4.1 Structure du projet
- 4.2 Authentification
- 4.3 Gestion des fichiers
- 4.4 Communication avec le backend

5. Sécurité et déploiement :

- 5.1 Sécurité
- 5.2 Déploiement
- 5.3 Haute disponibilité

6. Coûts et optimisation de l'infrastructure :

- 6.1 Coûts des services cloud
- 6.2 Optimisation des coûts
- 6.3 Choix d'architecture économique

7. Tests et résultats :

- 7.1 Tests réalisés
- 7.2 Résultats obtenus

8. Conclusion :

1. Introduction :

SUPFile est une application web de stockage cloud sécurisée développée dans le cadre du cursus SUPINFO. Inspiré des plateformes telles que Dropbox ou Google Drive, le projet a pour objectif de permettre aux utilisateurs de stocker, organiser, partager et gérer leurs fichiers de manière sécurisée à travers une interface moderne et intuitive.

Le frontend de SUPFile constitue la couche d'interaction principale entre l'utilisateur et les différents services du système. Il a été conçu selon une architecture client-serveur moderne afin d'offrir une expérience fluide, responsive et performante sur différents types d'appareils (ordinateurs, tablettes et smartphones).

L'interface utilisateur a été développée avec les technologies React 18, TypeScript et Vite, permettant la création d'une application dynamique, rapide et maintenable. Le frontend communique avec une API REST sécurisée développée en FastAPI, notamment pour la gestion de l'authentification, des fichiers, du partage de données et des fonctionnalités cloud.

Le projet intègre également plusieurs mécanismes de sécurité et de déploiement modernes :

- Authentification sécurisée via JWT et OAuth2 ;
- Communication sécurisée entre le client et le serveur ;
- Intégration d'un stockage cloud hybride ;
- Gestion des sessions utilisateurs ;
- Prise en charge des WebSockets pour certaines fonctionnalités temps réel ;
- Déploiement via une infrastructure distribuée utilisant Nginx, Gunicorn, FastAPI et Tailscale.

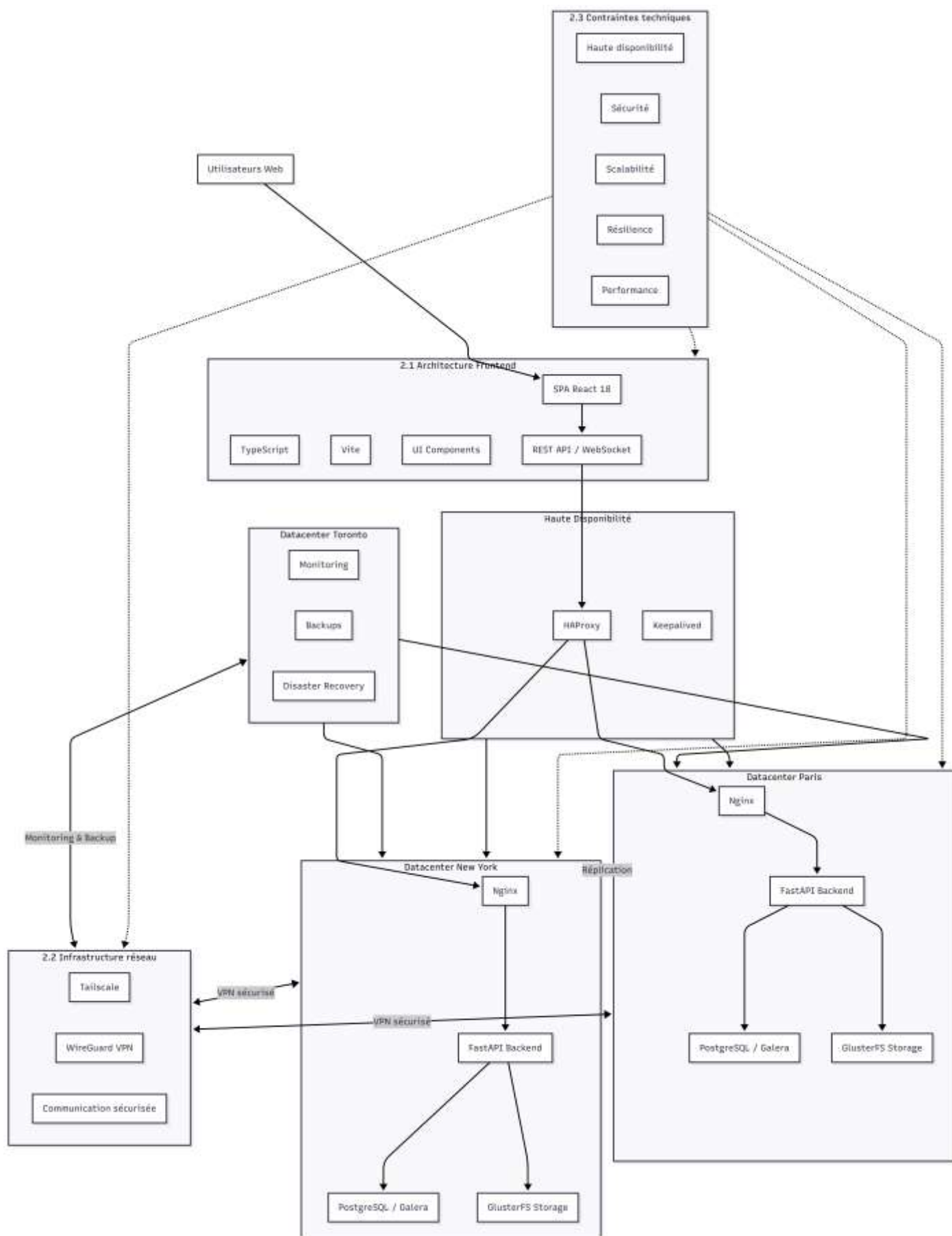
L'objectif principal du frontend est donc de fournir une interface ergonomique et sécurisée permettant aux utilisateurs d'interagir efficacement avec les services cloud de SUPFile tout en garantissant performance, stabilité et simplicité d'utilisation.

2. Présentation globale de l'architecture :

L'infrastructure de SUPFile repose sur une architecture distribuée multi-datacenter conçue pour garantir la haute disponibilité, la résilience et la sécurité des données.

Le système implémente un modèle Active/Active entre deux datacenters principaux situés à Paris et New York, complété par un troisième site à Toronto dédié aux sauvegardes, au monitoring et au plan de reprise d'activité (DRP).

L'objectif principal de cette architecture est d'assurer la continuité de service même en cas de panne d'un serveur, d'un composant réseau ou d'un datacenter complet, tout en maintenant les performances et l'intégrité des données utilisateurs.



L'infrastructure se compose de plusieurs couches techniques interconnectées :

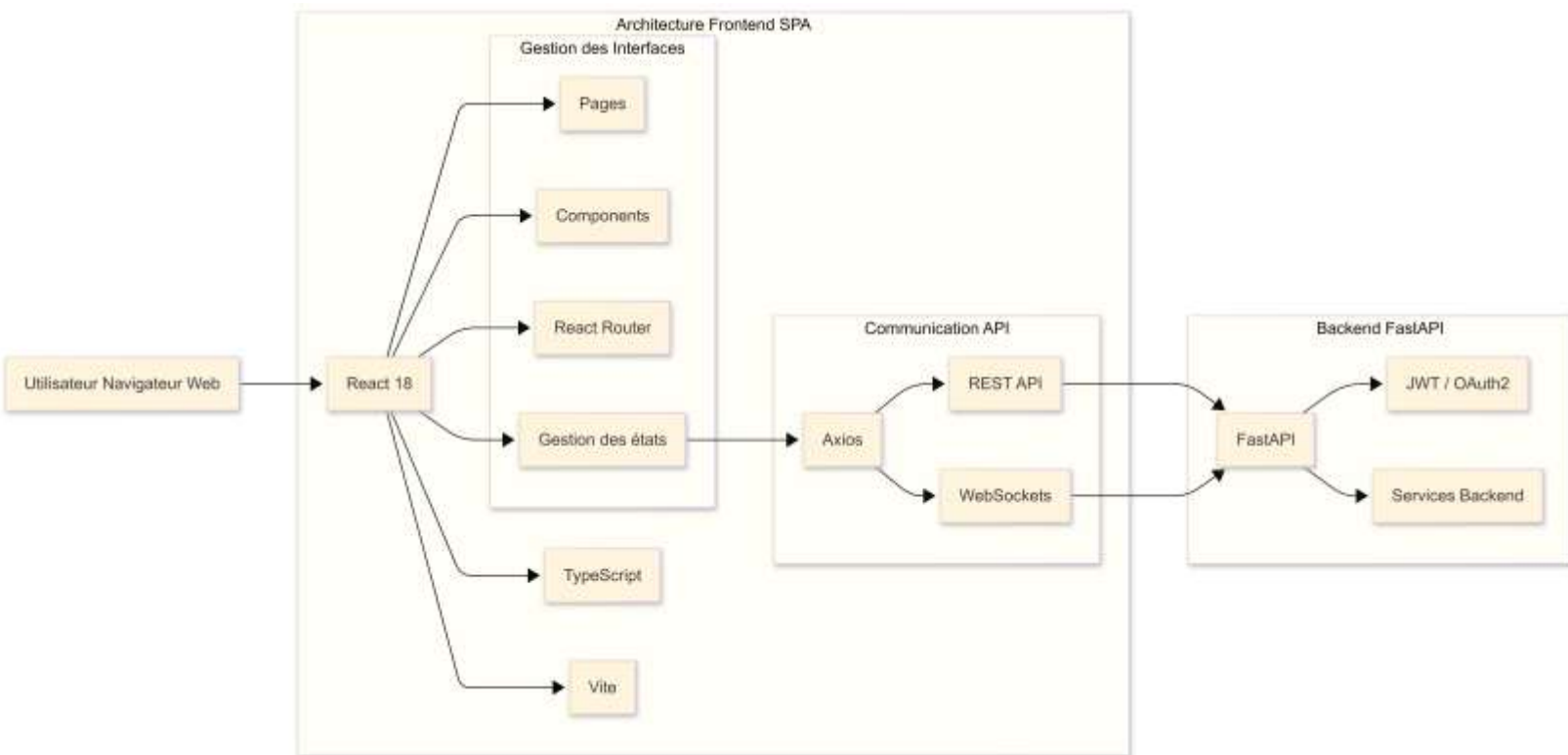
- Une couche frontend web accessible aux utilisateurs ;
- Une couche backend API FastAPI ;
- Une couche de répartition de charge haute disponibilité ;
- Une couche de stockage distribué ;
- Une couche base de données répliquée ;
- Une couche sécurité et supervision.

Les différents composants communiquent via un réseau privé sécurisé basé sur Tailscale/WireGuard permettant d'établir des connexions chiffrées entre les datacenters.

L'architecture repose sur les principes suivants :

- Suppression des points uniques de défaillance (SPOF) ;
- Réplication des services critiques ;
- Isolation réseau par VLAN ;
- Bascule automatique des services ;
- Supervision centralisée ;
- Tolérance aux pannes inter-sites.

2.1 Architecture frontend :



La couche applicative de SUPFile repose sur une architecture client-serveur moderne basée sur React, TypeScript et FastAPI.

Le frontend est déployé sur les serveurs web des datacenters Paris et New York afin de permettre :

- L'accès sécurisé aux fichiers ;
- L'authentification des utilisateurs ;
- Le partage de fichiers ;
- Les communications temps réel ;
- L'interaction avec les services cloud.

L'interface utilisateur communique avec l'API backend FastAPI via HTTPS et des requêtes REST sécurisées.

Le backend est exécuté sur plusieurs serveurs web afin d'assurer :

- La répartition de charge ;
- La continuité de service ;
- La montée en charge horizontale ;
- Le failover automatique.

Chaque datacenter possède :

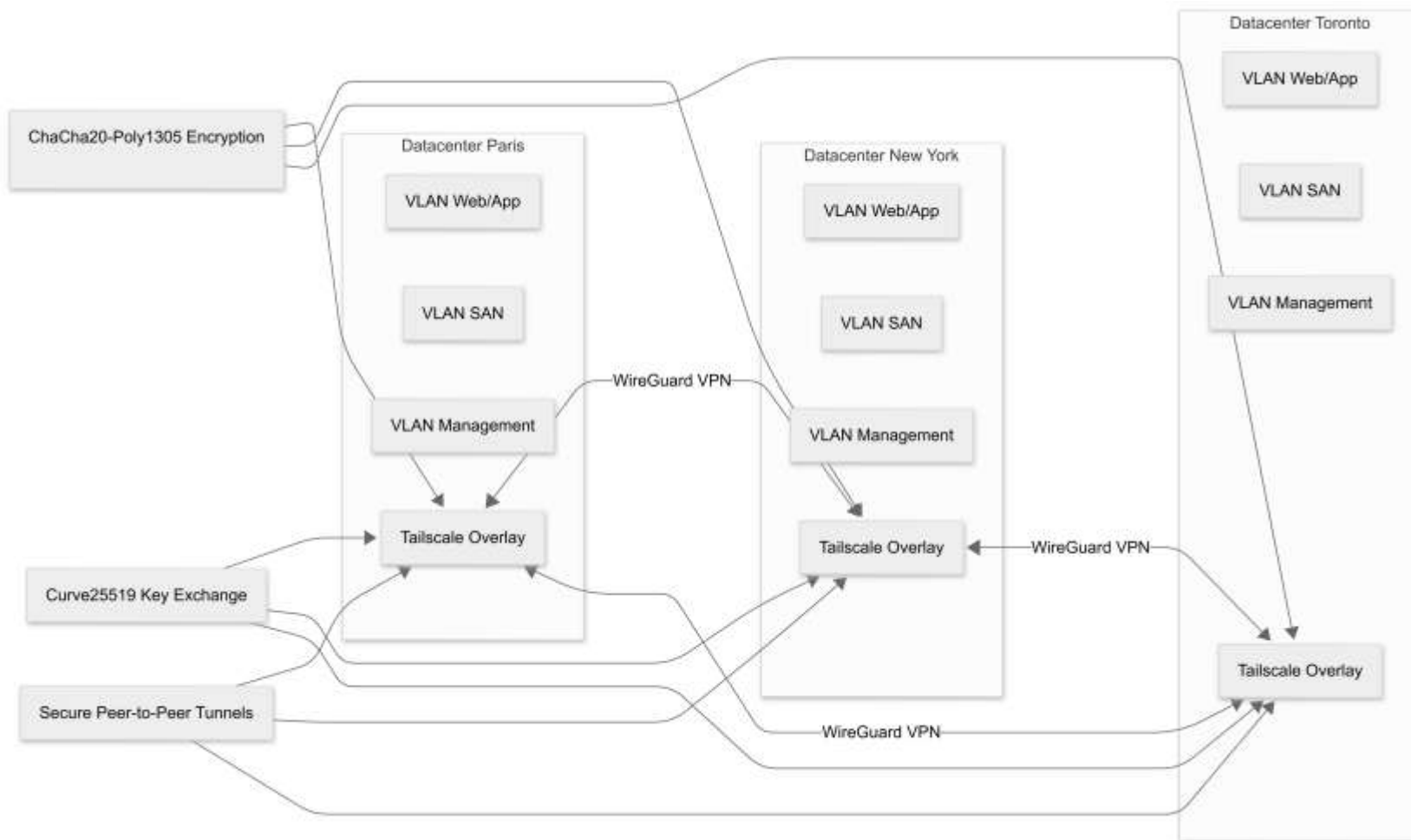
- Un load balancer HAProxy ;
- Deux serveurs web ;
- Un système de bascule automatique via Keepalived.

Le mécanisme HAProxy + Keepalived permet de garantir qu'en cas de panne d'un serveur web ou d'un load balancer, le trafic utilisateur est automatiquement redirigé vers un nœud fonctionnel sans interruption perceptible.

Le système fonctionne en mode Active/Active entre Paris et New York :

- Les deux datacenters traitent simultanément le trafic ;
- La charge est répartie dynamiquement ;
- Un datacenter peut reprendre automatiquement le trafic de l'autre en cas de panne.

2.2 Infrastructure réseau et communication inter-datacenter :



L'infrastructure réseau de SUPFile est segmentée en plusieurs VLANs isolés afin d'améliorer la sécurité et les performances.

Chaque datacenter possède :

- Un VLAN Web/App ;
- Un VLAN SAN ;
- Un VLAN Management ;
- Un réseau overlay Tailscale sécurisé.

Cette segmentation permet :

- D'isoler le trafic de stockage ;
- De protéger les serveurs critiques ;
- De limiter les mouvements latéraux en cas d'attaque ;
- De séparer les flux de production et d'administration.

Les communications entre Paris, New York et Toronto sont sécurisées grâce à Tailscale utilisant WireGuard comme protocole de tunnel VPN chiffré.

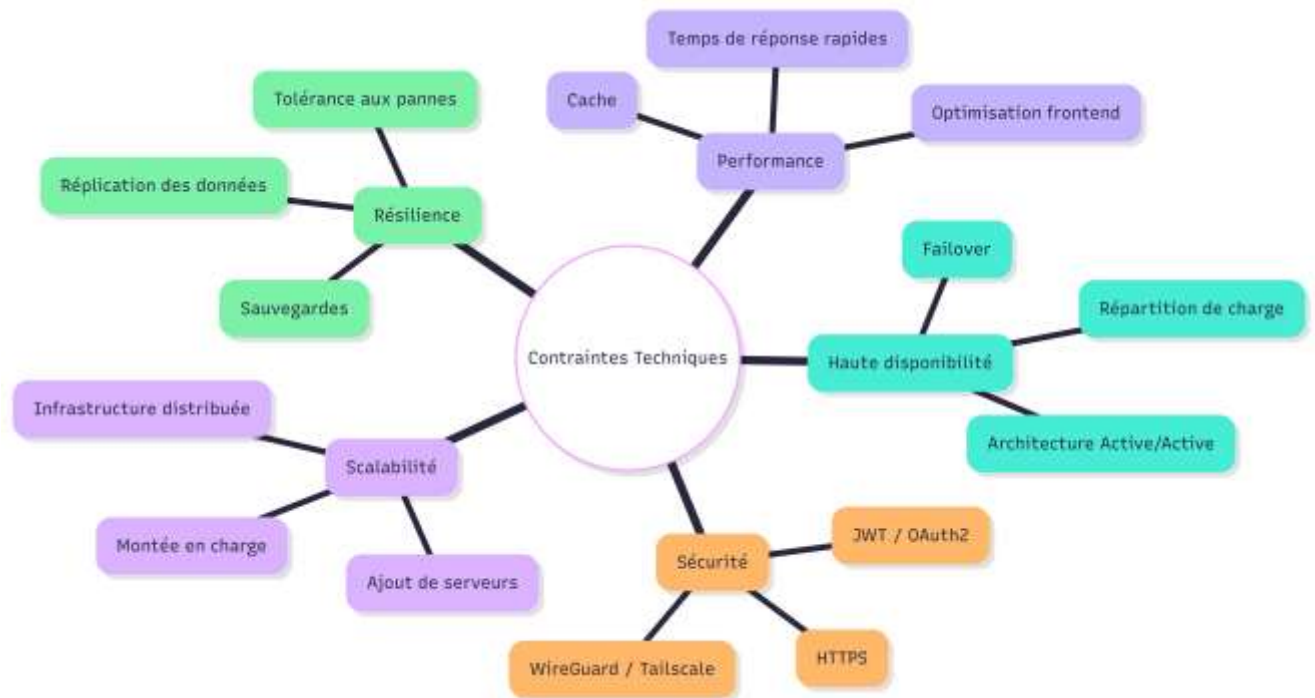
Les échanges inter-sites utilisent :

- Le chiffrement ChaCha20-Poly1305 ;
- L'échange de clés Curve25519 ;
- Des tunnels VPN peer-to-peer sécurisés.

Cette architecture garantit :

- La confidentialité des communications ;
- L'intégrité des données ;
- La sécurisation des répliques inter-datacenter.

2.3 Contraintes techniques:



L'architecture de SUPFile a été conçue afin de répondre à plusieurs contraintes techniques importantes liées aux applications cloud distribuées. Ces contraintes garantissent le bon fonctionnement du système, la sécurité des données ainsi que la continuité des services.

Haute disponibilité (HA) :

L'application doit rester accessible même en cas de panne d'un serveur ou d'un composant réseau. Pour répondre à cette exigence, l'infrastructure repose sur une architecture distribuée Active/Active entre plusieurs datacenters permettant d'assurer la continuité de service et la répartition de charge.

Réplication des données :

Les données utilisateurs et les fichiers stockés doivent être répliqués entre plusieurs serveurs afin d'éviter toute perte de données. Cette réplication permet également d'améliorer la tolérance aux pannes et d'assurer une meilleure disponibilité des fichiers.

Sécurité des communications :

Les échanges entre le frontend, le backend et les différents services cloud doivent être sécurisés. L'architecture utilise des protocoles sécurisés tels que HTTPS, JWT, OAuth2 et WireGuard/Tailscale afin de garantir la confidentialité, l'intégrité et l'authenticité des communications.

Scalabilité :

L'infrastructure doit pouvoir évoluer facilement afin de supporter un nombre croissant d'utilisateurs, de fichiers et de requêtes API. L'architecture distribuée permet d'ajouter de nouveaux serveurs ou services sans modifier profondément le fonctionnement global du système.

Résilience :

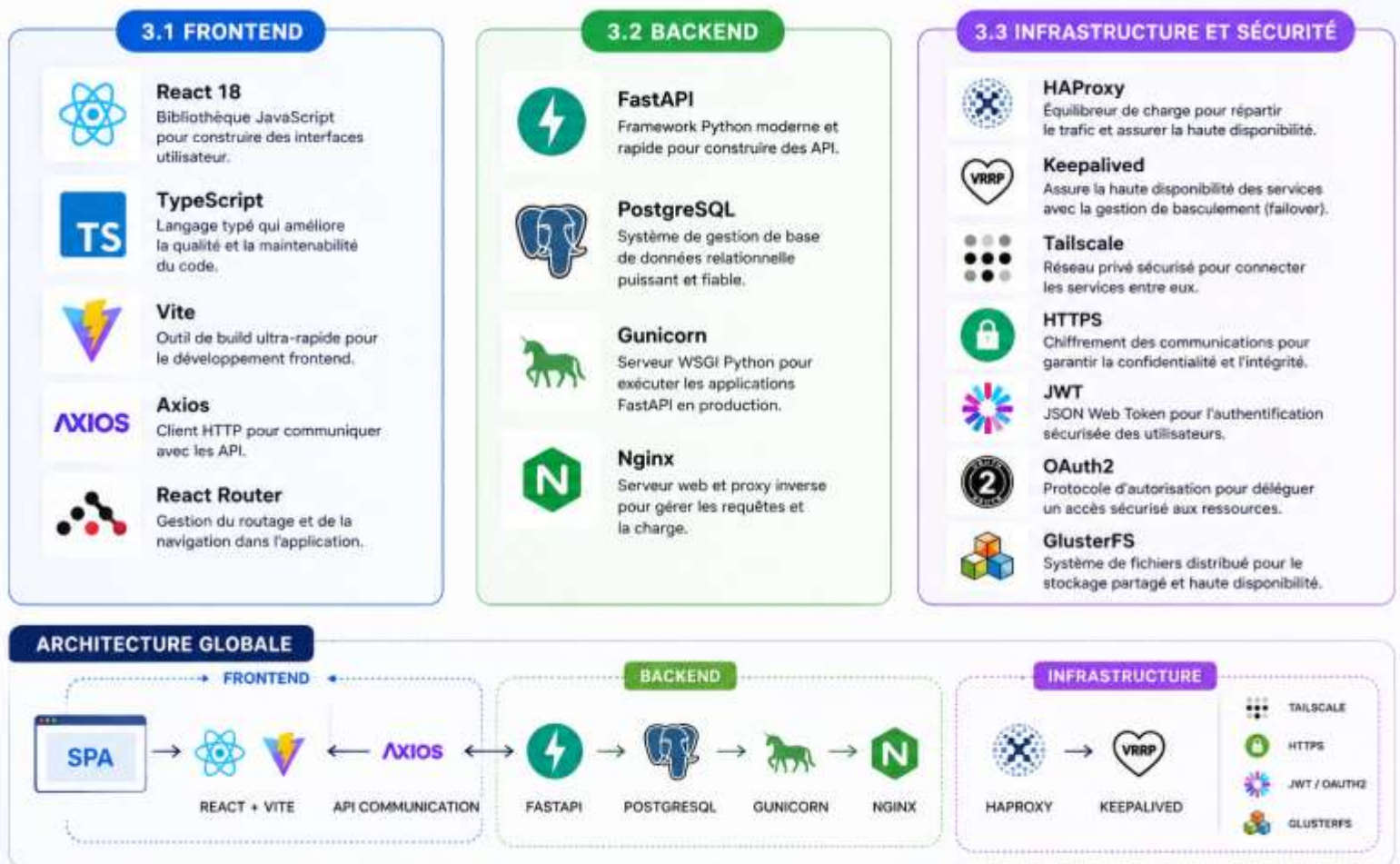
Le système doit être capable de continuer à fonctionner malgré certaines défaillances matérielles ou logicielles. Des mécanismes de redondance, de réplication et de bascule automatique (failover) ont été mis en place afin d'assurer la stabilité et la disponibilité de la plateforme.

Performance :

Le frontend doit offrir une expérience utilisateur fluide avec des temps de réponse rapides. L'utilisation de React, Vite, du cache navigateur et de mécanismes de proxy permet d'optimiser les performances globales de l'application.

3. Technologies utilisées :

Le projet SUPFile repose sur un ensemble de technologies modernes permettant de développer une application web performante, sécurisée et scalable. Les technologies choisies couvrent le développement frontend, le backend API ainsi que l'infrastructure réseau et les mécanismes de sécurité.



3.1 Frontend :

Le frontend de SUPFile a été développé à l'aide de technologies JavaScript modernes permettant de créer une interface utilisateur dynamique, responsive et performante.

❖ React 18 :

React 18 est utilisé comme bibliothèque principale pour la création de l'interface utilisateur. Son architecture basée sur les composants permet de structurer l'application de manière modulaire et réutilisable.

React offre plusieurs avantages :

- Rendu dynamique des interfaces ;
- Gestion efficace des composants ;
- Meilleure maintenabilité du code ;
- Amélioration des performances de rendu.

❖ **TypeScript :**

TypeScript est utilisé afin d'ajouter un typage statique au projet frontend. Cela permet :

- De réduire les erreurs de développement ;
- D'améliorer la lisibilité du code ;
- De faciliter la maintenance ;
- D'augmenter la robustesse de l'application.

❖ **Vite :**

Vite est utilisé comme outil de build et de développement frontend. Il permet :

- Un démarrage rapide du serveur de développement ;
- Une compilation optimisée ;
- Un rechargement instantané des modifications (Hot Reload) ;
- Une optimisation du bundle de production.

❖ **Axios :**

Axios est utilisé pour la communication entre le frontend et le backend FastAPI via des requêtes HTTP REST.

Cette bibliothèque permet :

- L'envoi de requêtes API ;
- La gestion des tokens JWT ;
- La gestion des erreurs réseau ;
- L'interception des requêtes et réponses HTTP.

❖ **React Router :**

React Router permet la gestion de la navigation dans l'application sous forme de SPA (Single Page Application).

Il est utilisé pour :

- La navigation entre les pages ;
- La gestion des routes protégées ;
- La redirection des utilisateurs ;
- La gestion des sessions utilisateur.

3.2 Backend

Le backend de SUPFile repose sur une architecture API moderne basée sur Python et FastAPI afin d'assurer la gestion des utilisateurs, des fichiers et des services cloud.

❖ FastAPI :

FastAPI est le framework principal utilisé pour développer l'API backend.

Il permet :

- La création d'API REST performantes ;
- La gestion des requêtes asynchrones ;
- L'intégration native de Swagger/OpenAPI ;
- Une excellente performance grâce à ASGI.

Le backend FastAPI gère notamment :

- L'authentification ;
- Les uploads et téléchargements ;
- Les métadonnées des fichiers ;
- Les WebSockets ;
- Les communications avec le stockage cloud.

❖ PostgreSQL :

PostgreSQL est utilisé comme système de gestion de base de données relationnelle.

Il permet le stockage :

- Des comptes utilisateurs ;

- Des permissions ;
- Des métadonnées des fichiers ;
- Des sessions ;
- Des informations de partage.

La base de données est intégrée dans une architecture distribuée afin d'assurer la réplication et la haute disponibilité.

❖ **Gunicorn :**

Gunicorn est utilisé comme serveur d'applications Python en production afin d'exécuter FastAPI avec plusieurs workers.

Il permet :

- Une meilleure stabilité ;
- Une gestion concurrente des requêtes ;
- Une amélioration des performances ;
- Une meilleure scalabilité du backend.

❖ **Nginx**

Nginx est utilisé comme reverse proxy devant le backend FastAPI et le frontend React.

Ses principales fonctions sont :

- La gestion du trafic HTTP ;
- Le reverse proxy API ;
- La gestion des WebSockets ;
- L'amélioration des performances ;
- La sécurisation des communications ;
- Le service des fichiers statiques frontend.

3.3 Infrastructure et sécurité :

L'infrastructure de SUPFile intègre plusieurs technologies réseau et sécurité permettant d'assurer la disponibilité, la résilience et la protection des communications.

❖ **HAProxy :**

HAProxy est utilisé comme load balancer afin de répartir le trafic entre plusieurs serveurs backend.

Il permet :

- La haute disponibilité ;
- La répartition de charge ;
- Le failover automatique ;
- L'amélioration des performances globales.

❖ **Keepalived :**

Keepalived est utilisé avec HAProxy afin de mettre en place un système de haute disponibilité basé sur des adresses IP virtuelles (VIP).

Il assure :

- La bascule automatique des services ;
- La tolérance aux pannes ;
- La continuité de service.

❖ **Tailscale :**

Tailscale est utilisé pour créer un réseau privé sécurisé entre les différents datacenters de l'infrastructure.

Basé sur WireGuard, il permet :

- Des tunnels VPN chiffrés ;
- Des communications inter-sites sécurisées ;
- Une administration simplifiée du réseau ;
- La sécurisation des répliquions inter-datacenter.

❖ **HTTPS :**

Le protocole HTTPS est utilisé afin de sécuriser les communications entre les utilisateurs, le frontend et le backend.

Il garantit :

- Le chiffrement des échanges ;
- L'intégrité des données ;

- La confidentialité des communications.

❖ **JWT :**

Les tokens JWT (JSON Web Tokens) sont utilisés pour gérer l'authentification des utilisateurs.

Ils permettent :

- L'authentification stateless ;
- La sécurisation des accès API ;
- La gestion des sessions utilisateur.

❖ **OAuth2 :**

OAuth2 est utilisé pour l'authentification externe via plusieurs fournisseurs :

- Google ;
- GitHub ;
- Microsoft Azure.

Cette technologie permet aux utilisateurs de se connecter de manière sécurisée sans partager directement leurs mots de passe avec l'application.

❖ **GlusterFS :**

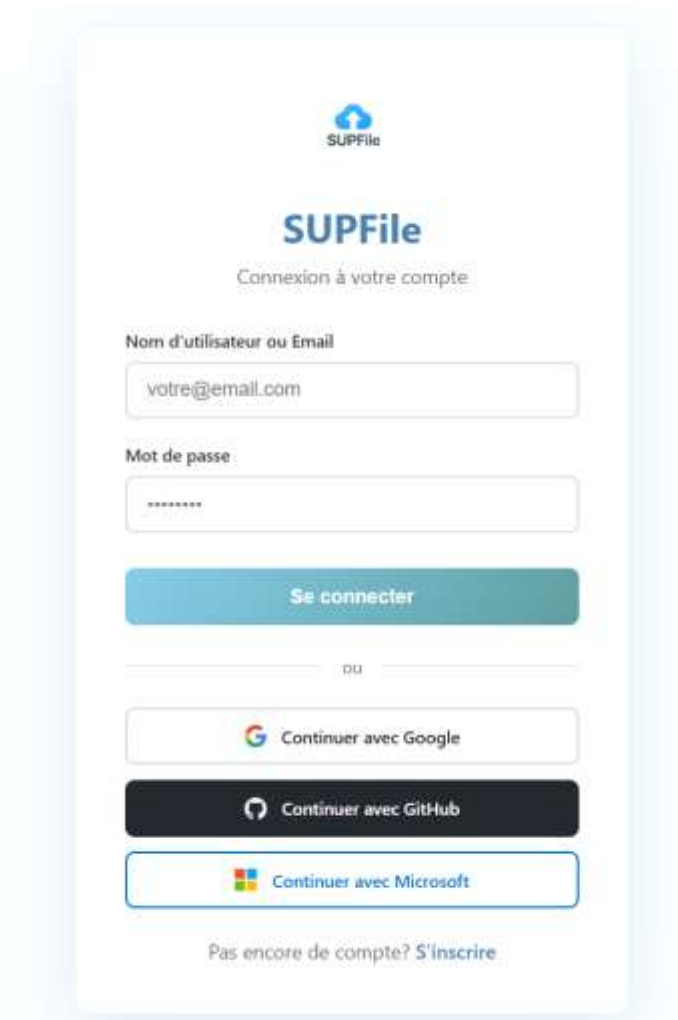
GlusterFS est utilisé comme système de stockage distribué afin d'assurer :

- La réplication des fichiers ;
- La tolérance aux pannes ;
- La haute disponibilité du stockage ;
- La distribution des données entre plusieurs serveurs.

4. Conception du Frontend :

Le frontend de SUPFile a été conçu dans le but de proposer une interface moderne, intuitive et performante permettant aux utilisateurs d'interagir facilement avec les services cloud de la plateforme. L'architecture frontend repose sur une organisation modulaire facilitant la maintenance, l'évolution du projet et l'intégration des différentes fonctionnalités.

L'application est développée sous forme de SPA (Single Page Application) avec React et TypeScript afin d'améliorer la fluidité de navigation et les performances globales.



The image shows a login interface for an application named SUPFile. At the top, there is a logo consisting of a blue cloud icon with the text 'SUPFile' below it. Below the logo, the text 'Connexion à votre compte' is displayed. The form includes two input fields: 'Nom d'utilisateur ou Email' with the placeholder text 'votre@email.com', and 'Mot de passe' with a masked password '*****'. A blue 'Se connecter' button is positioned below the password field. A horizontal line with the word 'ou' in the center separates the login section from the social login options. These options include 'Continuer avec Google' (with the Google logo), 'Continuer avec GitHub' (with the GitHub logo), and 'Continuer avec Microsoft' (with the Microsoft logo). At the bottom, there is a link that says 'Pas encore de compte? S'inscrire'.


SUPFile

Connexion à votre compte

Nom d'utilisateur ou Email

Mot de passe

Se connecter

ou

 Continuer avec Google

 Continuer avec GitHub

 Continuer avec Microsoft

Pas encore de compte? [S'inscrire](#)

4.1 Structure du projet :



Le projet frontend est organisé de manière modulaire afin de séparer clairement les différentes responsabilités de l'application.

L'arborescence principale du projet contient notamment :

- **Les pages de navigation ;**
- **Les composants réutilisables ;**
- **Les services API ;**

- **Les fichiers de configuration ;**
- **Les hooks React ;**
- **Les contextes de gestion des états.**

❖ **Organisation des dossiers :**

Le dossier ``src`` contient l'ensemble du code source de l'application frontend.

Il est structuré autour de plusieurs dossiers principaux :

- ``pages/`` : contient les différentes pages de l'application ;
- ``components/`` : contient les composants réutilisables de l'interface ;
- ``services/`` : contient les appels API et la logique de communication backend ;
- ``context/`` : gestion des états globaux et des sessions utilisateur ;
- ``hooks/`` : hooks React personnalisés ;
- ``styles/`` : fichiers CSS et styles globaux ;
- ``utils/`` : fonctions utilitaires et helpers.

Cette organisation améliore :

- La lisibilité du projet ;
- La réutilisation du code ;
- La maintenabilité ;
- La scalabilité du frontend.

❖ **Components :**

Les composants React permettent de construire l'interface utilisateur de manière modulaire.

Chaque composant possède une responsabilité précise, par exemple :

- Barre de navigation ;
- Formulaire de connexion ;
- Gestionnaire de fichiers ;
- Fenêtres modales ;
- Tableaux de fichiers ;
- Notifications utilisateur.

Cette approche facilite :

- La réutilisation des interfaces ;
- Les mises à jour ;
- Les tests unitaires.

❖ **Pages :**

Les pages représentent les principales vues accessibles par les utilisateurs :

- Page de connexion ;
- Tableau de bord ;
- Espace de stockage ;
- Gestion des fichiers ;
- Profil utilisateur ;
- Partage de fichiers.

La navigation entre les pages est gérée par React Router.

❖ **Services API :**

Les services API centralisent tous les appels HTTP vers le backend FastAPI.

Ils permettent :

- D'isoler la logique réseau ;
- De gérer les tokens JWT ;
- De simplifier les requêtes API ;
- De centraliser la gestion des erreurs.

❖ **Gestion des états :**

La gestion des états permet de partager certaines données entre plusieurs composants de l'application.

Le frontend utilise :

- React Context ;

- Les hooks React ;
- Le stockage local du navigateur.

Les états gérés comprennent notamment :

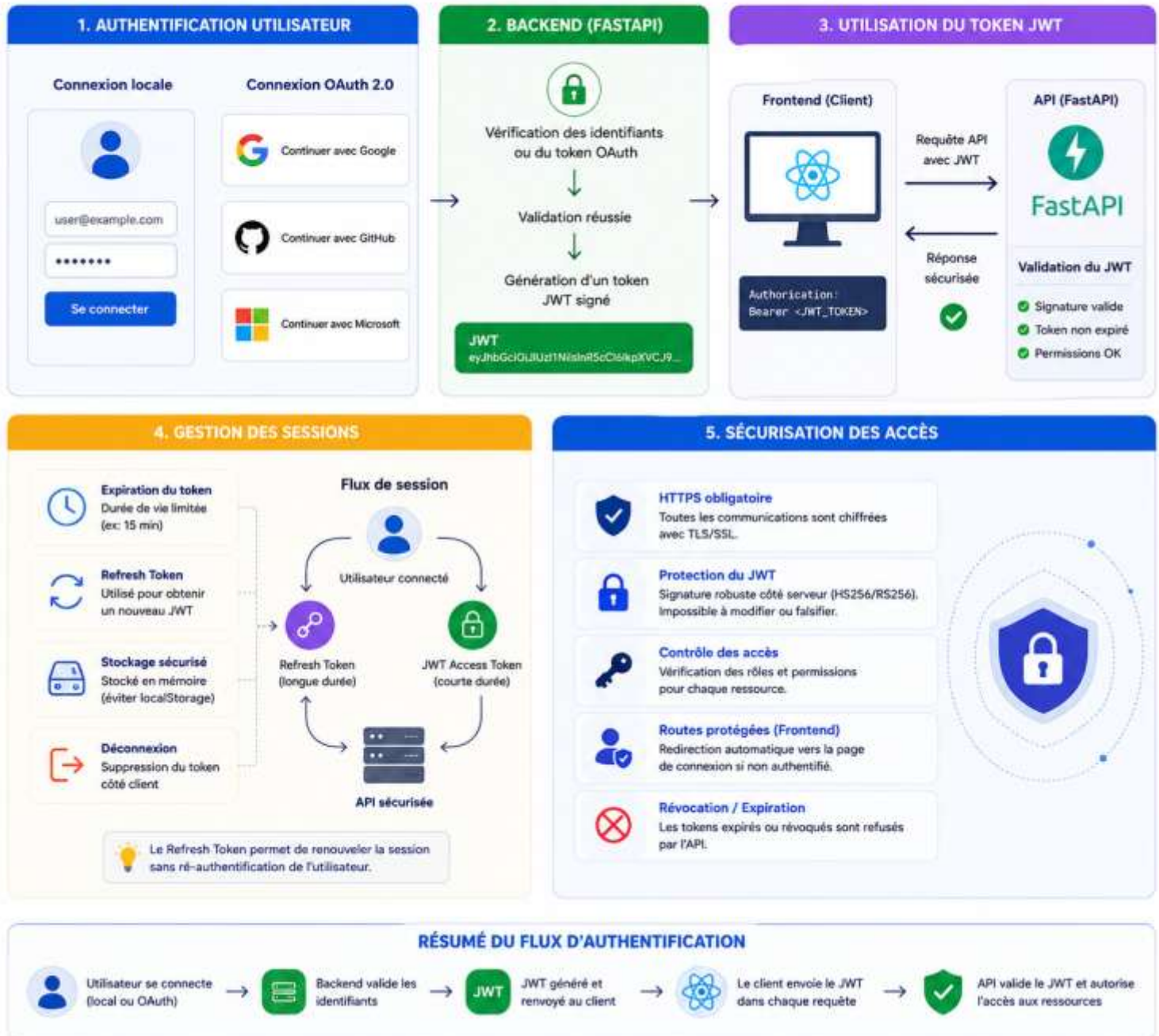
- Les informations utilisateur ;
- Les sessions d'authentification ;
- Les fichiers affichés ;
- Les notifications ;
- Les états de chargement.

4.2 Authentification :



4.2 AUTHENTIFICATION

Sécurisée • Flexible • Centralisée



Le système d'authentification de SUPFile a été conçu afin de sécuriser l'accès aux données utilisateurs tout en simplifiant l'expérience de connexion.

❖ **JWT :**

L'authentification principale repose sur l'utilisation de JWT (JSON Web Tokens).

Après connexion :

- Le backend génère un token sécurisé ;
- Le frontend stocke temporairement le token ;
- Les requêtes API authentifiées utilisent ce token dans les headers HTTP.

Cette approche permet :

- Une authentification stateless ;
- Une meilleure scalabilité ;
- Une sécurisation des accès API.

❖ **OAuth Google/GitHub/Microsoft :**

Le frontend intègre également l'authentification OAuth2 via plusieurs fournisseurs externes :

- Google ;
- GitHub ;
- Microsoft Azure.

Cette fonctionnalité permet :

- Une connexion simplifiée ;
- Une meilleure expérience utilisateur ;
- Une réduction de la gestion des mots de passe.

Le processus OAuth redirige les utilisateurs vers le fournisseur d'authentification avant de retourner vers le backend FastAPI.

❖ **Gestion des sessions :**

Le frontend gère les sessions utilisateur à travers :

- * le stockage des tokens ;
- * la vérification des expirations ;
- * la reconnexion automatique ;
- * les redirections vers la page de connexion.

Les sessions sont synchronisées avec le backend afin de sécuriser les accès.

❖ **Sécurisation des accès**

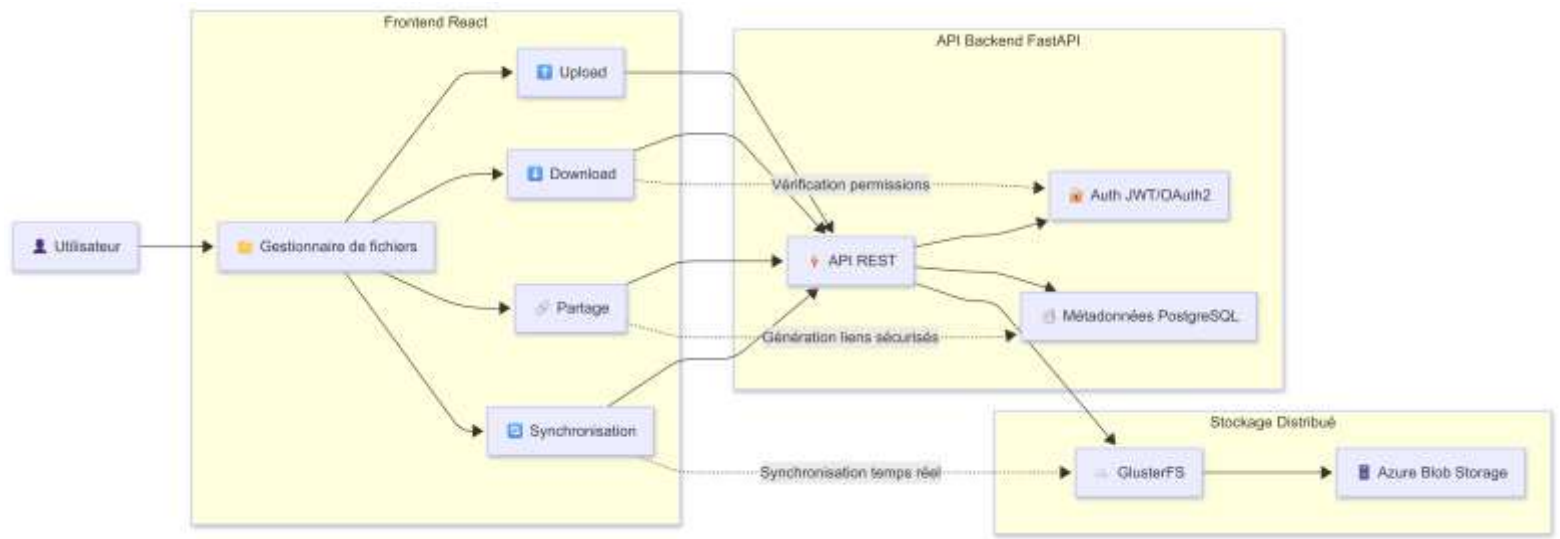
Certaines routes frontend sont protégées afin de limiter l'accès aux utilisateurs authentifiés.

Le frontend vérifie :

- La présence du token JWT ;
- La validité de la session ;
- Les permissions utilisateur.

Les utilisateurs non authentifiés sont automatiquement redirigés vers la page de connexion.

4.3 Gestion des fichiers :



La gestion des fichiers constitue la fonctionnalité principale de SUPFile.

Le frontend permet aux utilisateurs :

- D'envoyer des fichiers ;
- De télécharger des documents ;
- De partager des fichiers ;
- De synchroniser les données avec le backend.

❖ Upload

Le système d'upload permet :

- L'envoi de fichiers vers le stockage cloud ;
- La gestion des tailles maximales ;
- Le suivi de progression ;
- La validation des extensions autorisées.

Les fichiers sont transmis au backend via des requêtes HTTP sécurisées.

❖ Download

Les utilisateurs peuvent télécharger leurs fichiers directement depuis l'interface web.

Le frontend gère :

- Les liens sécurisés ;
- Le déclenchement des téléchargements ;
- Les permissions d'accès.

❖ **Partage**

SUPFile permet le partage sécurisé de fichiers entre utilisateurs.

Le frontend fournit :

- Des liens de partage ;
- La gestion des permissions ;
- L'interface de partage utilisateur.

❖ **Synchronisation**

Les données affichées dans l'interface sont synchronisées avec le backend via les API REST et les WebSockets.

Cette synchronisation permet :

- L'actualisation des fichiers ;
- La mise à jour des notifications ;
- L'affichage des nouveaux contenus.

 **SUPFile**

 Accéder à mes fichiers

 Mes fichiers

 Fichiers partagés

 Aide & Commentaires

 Corbeille

 À propos

 Se déconnecter



mevengueingotfrank

mevengueingotfrank@gmail.com

 Déconnexion

Statistiques

 **5**
Dossiers

 **0**
Fichiers à la poubelle

 **0**
Fichiers totaux

Espace de stockage

 **15.26 MB**
Espace occupé

 **99.99 GB**
Espace disponible

 **0.0%**
Quota utilisé

 **100 GB**
Quota total

Structure des dossiers

 **Fichier 1**

2 fichiers = 0 sous-dossiers

 **Fichier 2**

3 fichiers = 0 sous-dossiers

 **Musique**

4 fichiers = 0 sous-dossiers

 **Vidéo**

6 fichiers = 0 sous-dossiers

 **Audio**

2 fichiers = 0 sous-dossiers

Chercher

Chercher

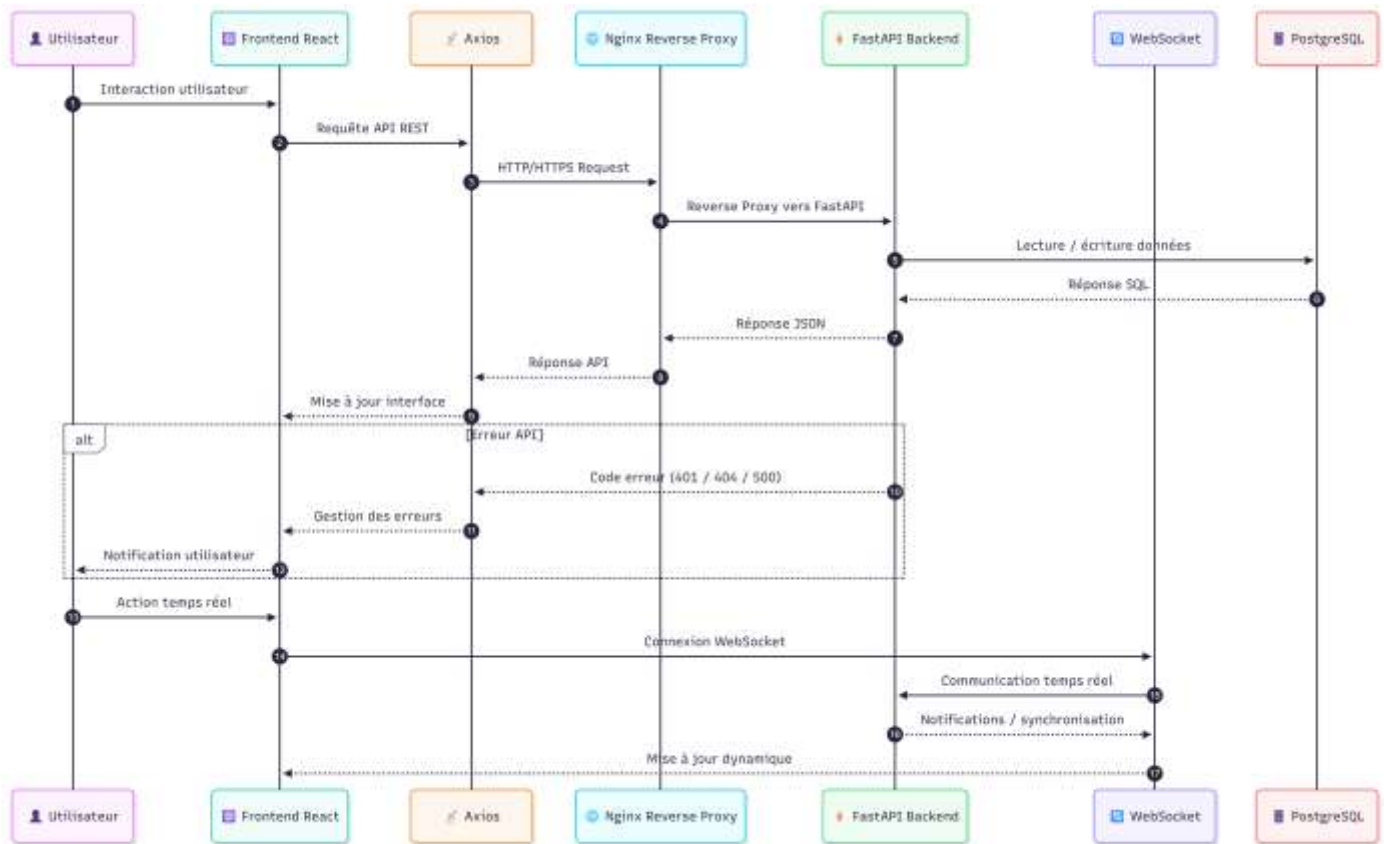
Chercher

Chercher

Chercher

Masquer les statistiques

4.4 Communication avec le backend :



Le frontend communique avec le backend FastAPI via une architecture REST sécurisée.

❖ API REST

Les fonctionnalités principales utilisent des endpoints REST :

- Authentification ;
- Gestion des utilisateurs ;
- upload et téléchargement ;
- Gestion des fichiers ;
- Partage.

Les échanges utilisent le format JSON.

❖ **Axios**

Axios est utilisé comme client HTTP principal.

Il permet :

- L'envoi des requêtes API ;
- La gestion des headers JWT ;
- L'interception des erreurs ;
- La gestion centralisée des requêtes.

❖ **Gestion des erreurs**

Le frontend intègre une gestion des erreurs afin d'améliorer l'expérience utilisateur.

Les erreurs gérées comprennent :

- Erreurs réseau ;
- Erreurs d'authentification ;
- Expiration des sessions ;
- Erreurs serveur ;
- Erreurs de permissions.

Des messages utilisateur sont affichés afin d'informer clairement l'utilisateur des problèmes rencontrés.

❖ **WebSockets**

Les WebSockets sont utilisés pour certaines fonctionnalités temps réel.

Ils permettent :

- La synchronisation instantanée ;
- Les notifications ;
- La mise à jour dynamique des données.

Le frontend gère également :

- La reconnexion automatique ;
- La détection des pertes de connexion ;
- La stabilisation des sessions WebSocket.

❖ Reverse Proxy Nginx

Le frontend communique avec le backend à travers un reverse proxy Nginx.

Cette architecture permet :

- De centraliser les accès ;
- D'améliorer la sécurité ;
- De supprimer les problèmes CORS ;
- De stabiliser les communications frontend/backend ;
- De gérer correctement les WebSockets et les API REST.

5 Sécurité et déploiement :

La sécurité et le déploiement constituent des éléments essentiels de l'architecture de SUPFile. Le frontend a été conçu afin de garantir la protection des données utilisateurs, la sécurisation des communications ainsi qu'une disponibilité élevée des services. L'infrastructure de déploiement repose sur une architecture distribuée permettant d'assurer la stabilité, la résilience et la continuité de service de l'application.

5. SÉCURITÉ



HTTPS

- Chiffrement TLS/SSL
- Communications sécurisées



PROTECTION DES TOKENS JWT

- Tokens signés et vérifiés
- Expiration des tokens
- Stockage sécurisé (localStorage)



SÉCURISATION DES APPELS API

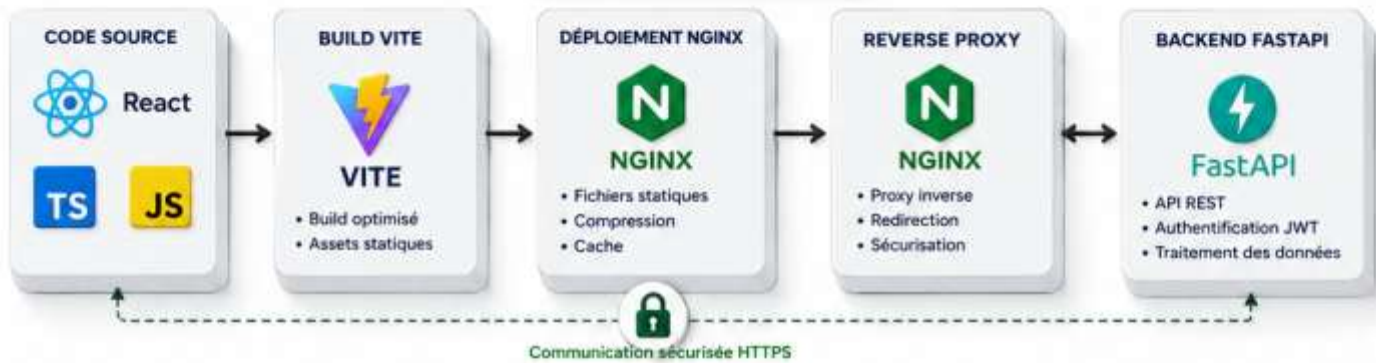
- Appels via HTTPS
- En-têtes sécurisés (Authorization)
- Validation côté serveur



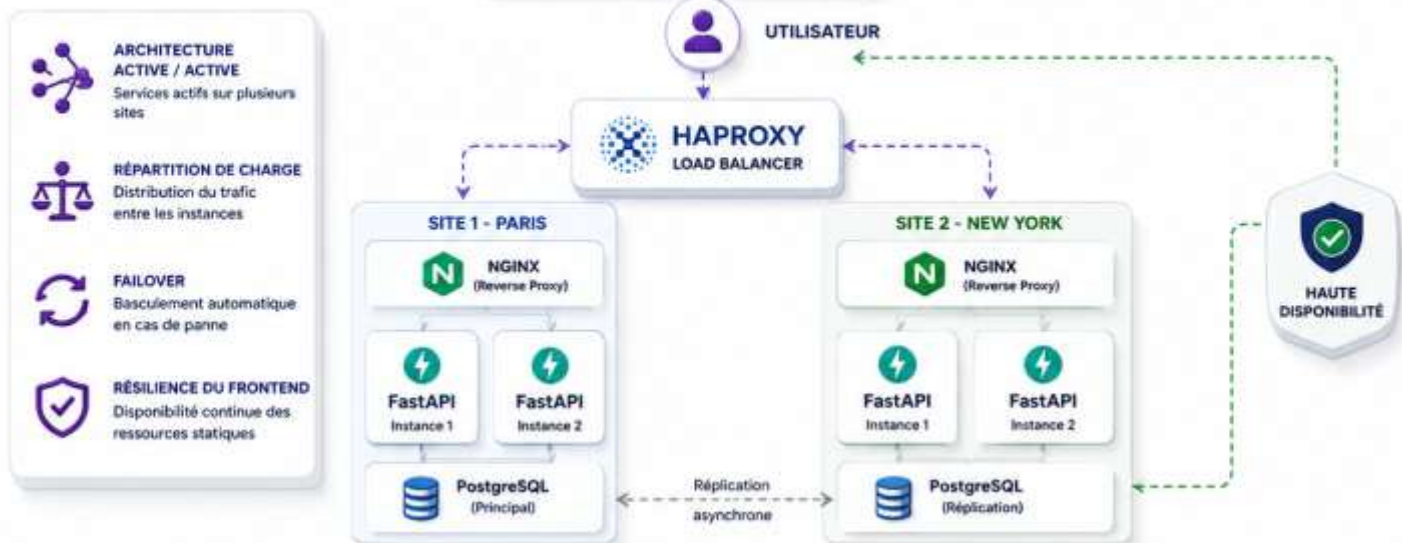
GESTION DES PERMISSIONS

- Rôles et permissions utilisateurs
- Contrôle d'accès (RBAC)
- Protection des routes et ressources

5.2 DÉPLOIEMENT



5.3 HAUTE DISPONIBILITÉ



5.1 Sécurité :

Le frontend de SUPFile intègre plusieurs mécanismes de sécurité destinés à protéger les utilisateurs, les données et les communications avec le backend.

❖ HTTPS

Toutes les communications entre le navigateur utilisateur et les services backend sont sécurisées via le protocole HTTPS.

Le chiffrement HTTPS permet :

- De protéger les données échangées ;
- D'éviter les interceptions réseau ;
- De garantir l'intégrité des communications ;
- De sécuriser les connexions utilisateurs.

L'utilisation de HTTPS est particulièrement importante pour :

- L'authentification ;
- Les échanges de tokens JWT ;
- Les uploads de fichiers ;
- Les communications API.

❖ Protection des tokens JWT

Le système d'authentification repose sur l'utilisation de JWT (JSON Web Tokens).

Le frontend gère les tokens afin de :

- Authentifier les utilisateurs ;
- Sécuriser les accès API ;
- Maintenir les sessions utilisateur.

Des mécanismes de protection ont été mis en place afin de :

- Limiter les accès non autorisés ;
- Gérer l'expiration des sessions ;
- Empêcher l'utilisation de tokens invalides.

Les tokens sont automatiquement transmis dans les headers HTTP des requêtes API sécurisées.

❖ **Sécurisation des appels API**

Les communications entre le frontend React et le backend FastAPI sont sécurisées via :

- HTTPS ;
- JWT ;
- reverse proxy Nginx ;
- Contrôle des accès API.

Les appels API utilisent des endpoints sécurisés nécessitant une authentification valide avant l'accès aux ressources sensibles.

Le frontend vérifie également :

- Les réponses serveur ;
- Les erreurs d'authentification ;
- Les permissions utilisateur ;
- Les expirations de session.

❖ **Gestion des permissions**

Le système de permissions permet de contrôler l'accès aux différentes fonctionnalités de l'application.

Le frontend applique des restrictions selon :

- Le statut d'authentification ;
- Les permissions utilisateur ;
- Les droits de partage des fichiers.

Certaines pages et fonctionnalités sont uniquement accessibles aux utilisateurs authentifiés.

5.2 Déploiement :

Le frontend de SUPFile a été déployé à l'aide d'une architecture moderne permettant d'assurer de bonnes performances, une stabilité élevée et une communication sécurisée avec le backend.

❖ **Build Vite**

Le frontend est compilé en mode production grâce à Vite.

Le processus de build permet :

- L'optimisation des fichiers JavaScript ;
- La réduction de la taille des bundles ;
- L'amélioration des performances de chargement ;
- La génération des fichiers statiques de production.

Les fichiers générés sont ensuite déployés sur les serveurs web Nginx.

❖ **Déploiement Nginx**

Nginx est utilisé comme serveur web principal afin d'héberger les fichiers statiques du frontend React.

Il permet :

- De servir l'application frontend ;
- D'améliorer les performances ;
- De gérer les connexions HTTP/HTTPS ;
- D'optimiser le cache des fichiers statiques.

Nginx joue également un rôle important dans la sécurisation de l'infrastructure grâce à sa fonction de reverse proxy.

❖ **Reverse Proxy**

Le reverse proxy Nginx permet de centraliser les communications entre le frontend et le backend FastAPI.

Cette architecture présente plusieurs avantages :

- Suppression des problèmes CORS ;
- Sécurisation des endpoints backend ;
- Meilleure gestion des WebSockets ;
- Optimisation des performances réseau ;
- Réduction des accès directs aux services internes.

Les utilisateurs accèdent uniquement à une seule adresse publique tandis que les services backend restent isolés au sein du réseau interne.

❖ **Communication avec FastAPI**

Le frontend communique avec le backend FastAPI via des requêtes API REST sécurisées.

Les échanges permettent :

- L'authentification des utilisateurs ;
- La gestion des fichiers ;
- Les uploads et téléchargements ;
- Les opérations de partage ;
- Les communications temps réel.

Le backend FastAPI est exécuté avec Gunicorn et Uvicorn afin d'assurer une meilleure stabilité et une meilleure gestion des connexions simultanées.

5.3 Haute disponibilité :

L'infrastructure de SUPFile a été conçue afin d'assurer une haute disponibilité des services frontend et backend.

❖ **Architecture Active/Active**

Le système repose sur une architecture Active/Active entre plusieurs datacenters situés à Paris et New York.

Cette architecture permet :

- La répartition simultanée du trafic ;
- L'amélioration des performances ;
- La continuité de service ;
- La réduction des points uniques de défaillance.

Les deux sites peuvent traiter les requêtes utilisateurs simultanément.

❖ **Répartition de charge**

Le trafic réseau est distribué grâce à des mécanismes de load balancing utilisant HAProxy.

La répartition de charge permet :

- D'équilibrer le trafic entre plusieurs serveurs ;
- D'éviter la surcharge d'un nœud ;
- D'améliorer les performances globales ;
- D'augmenter la capacité de traitement.

❖ **Failover**

Des mécanismes de bascule automatique (failover) ont été mis en place afin d'assurer la continuité des services en cas de panne.

Keepalived permet :

- La gestion des IP virtuelles ;
- La détection des pannes ;
- Le basculement automatique vers un serveur secondaire.

Ainsi, si un serveur devient indisponible, le trafic est automatiquement redirigé vers un autre nœud fonctionnel.

❖ **Résilience du frontend**

Le frontend a été conçu pour rester opérationnel malgré certaines défaillances réseau ou serveur.

Plusieurs mécanismes contribuent à cette résilience :

- Architecture distribuée ;
- Réplication des services ;
- Reverse proxy Nginx ;
- Gestion des erreurs réseau ;
- Reconnexion automatique des WebSockets ;
- Système de cache navigateur et Service Worker.

Ces mécanismes permettent d'améliorer la stabilité globale de l'application et l'expérience utilisateur.

6 Coûts et optimisation de l'infrastructure :

L'infrastructure SUPFile a été conçue selon une approche hybride combinant des ressources cloud publiques et une infrastructure virtualisée locale basée sur VirtualBox et Vagrant. Cette architecture permet de réduire fortement les coûts tout en conservant un environnement proche des infrastructures professionnelles utilisées en entreprise.

Le projet repose sur trois datacenters logiques :

- Paris ;
- New York ;
- Toronto.

Les environnements Paris et New York utilisent des machines virtuelles VirtualBox simulant une infrastructure multi-datacenter distribuée, tandis que certains services cloud externes sont utilisés pour les composants publics et les services internet.

Cette approche permet :

- De réduire les dépenses d'hébergement ;
- De tester des architectures complexes à faible coût ;
- De conserver un haut niveau de flexibilité ;
- De simuler une infrastructure de production réaliste.

6.2 Coûts des services cloud :

Même si une partie importante de l'infrastructure repose sur des machines virtuelles locales, certains services cloud restent nécessaires afin de garantir l'accessibilité publique de la plateforme.

❖ Azure Blob Storage

Azure Blob Storage peut être utilisé pour :

- Le stockage externe ;
- Les sauvegardes cloud ;
- L'archivage des données ;
- La réplication distante.

Dans le cadre du POC, l'utilisation reste limitée afin de réduire les coûts.

Estimation mensuelle :

- Stockage : 5 à 15 € / mois ;
- Trafic réseau : variable selon l'utilisation.

❖ Hébergement Frontend

Le frontend React peut être hébergé sur plusieurs plateformes :

- Vercel ;
- Azure Static Web Apps ;
- Nginx local ;
- VM VirtualBox exposée via reverse proxy.

Le projet utilise principalement :

- Vite ;
- Nginx ;
- Hébergement sur VM Linux.

Le coût est donc fortement réduit grâce à l'utilisation des machines virtuelles locales.

Estimation :

- Vercel Hobby : gratuit ;
- Vercel Pro : environ 20 € / mois ;
- Hébergement local : coût quasi nul hors consommation électrique.

❖ Services réseau et DNS

L'infrastructure nécessite :

- Un nom de domaine ;
- Un système DNS ;
- Des tunnels VPN sécurisés.

Les coûts principaux concernent :

- OVHCloud ;
- Cloudflare ;
- Certificats HTTPS.

Estimation mensuelle :

- Domaine .fr : ~1 à 2 € / mois ;
- Cloudflare : gratuit dans le POC ;
- Certificats Let's Encrypt : gratuits.

❖ Utilisation des ressources cloud

Les services cloud restent volontairement limités afin de privilégier :

- L'optimisation budgétaire ;
- L'apprentissage technique ;
- La maîtrise de l'infrastructure.

Le projet privilégie une logique de :

- Cloud hybride ;
- Infrastructure auto-hébergée ;
- Virtualisation locale.

6.3 Optimisation des coûts :

L'architecture SUPFile a été pensée afin d'obtenir le meilleur compromis entre :

- Coût ;
- Sécurité ;
- Disponibilité ;
- Évolutivité.

❖ Infrastructure hybride cloud/on-premise

Le projet combine :

- Des services cloud publics ;
- Des machines virtuelles locales ;
- Des solutions open-source.

Cette approche hybride permet :

- De limiter les coûts cloud ;
- De conserver un contrôle sur l'infrastructure ;
- De réduire la dépendance aux fournisseurs externes.

❖ Utilisation de machines virtuelles

L'utilisation de VirtualBox et Vagrant permet de simuler :

- Plusieurs datacenters ;
- Des VLANs ;
- Des systèmes de stockage ;
- Des architectures HA.

Le coût matériel reste très faible comparé à une infrastructure physique réelle.

Le POC fonctionne avec :

- 17+ machines virtuelles ;
- Plusieurs VLANs isolés ;
- Une architecture multi-sites ;
- Une réplication inter-datacenter.

❖ Déploiement distribué à faible coût

L'utilisation de :

- Tailscale ;
- GlusterFS ;
- HAProxy ;
- Keepalived ;
- PostgreSQL Galera ;
- ProxySQL ;

Permet de reproduire des architectures d'entreprise avancées sans nécessiter d'équipements professionnels coûteux.

L'ensemble de l'infrastructure reste déployable sur :

- Des ordinateurs personnels ;
- Des serveurs standards ;
- Des environnements cloud peu coûteux.

❖ **Scalabilité progressive**

L'architecture a été conçue pour évoluer progressivement selon les besoins.

Cela permet :

- D'ajouter de nouveaux serveurs ;
- D'augmenter la capacité de stockage ;
- D'améliorer les performances ;
- D'étendre le nombre d'utilisateurs.

L'approche modulaire évite de devoir investir massivement dès le départ.

❖ **Estimation budgétaire du POC**

ÉLÉMENT	TECHNOLOGIE	COÛT ESTIMÉ
1  Frontend Hosting	 Vercel 	€ Gratuit à 20 €/mois
2  Domaine + DNS	 OVHcloud 	€ 1 à 5 €/mois
3  Azure Blob Storage	 Microsoft Azure	€ 5 à 15 €/mois
4  VPN Inter-DC	 tailscale	€ Gratuit
5  Monitoring	 Prometheus  Grafana	€ Gratuit
6  Reverse Proxy	 HAPROXY 	€ Gratuit
7  Base de données	 PostgreSQL  GALERA	€ Gratuit
8  Virtualisation	 VirtualBox  VAGRANT	€ Gratuit
9  Stockage distribué	 GLUSTER	€ Gratuit

❖ Coût mensuel estimé du POC

Cette approche hybride permet donc de développer et tester une infrastructure cloud distribuée complexe tout en maintenant des coûts très faibles adaptés à un projet académique et de recherche.



COÛT MENSUEL ESTIMÉ DU POC

1. INFRASTRUCTURE POC HYBRIDE



~10 à 40 € / mois

Solution économique
Idéale pour un POC

Élément	Coût estimé / mois
▲ Frontend (Vercel / Nginx)	0 - 20 €
🌐 Domaine + DNS (OVH + Cloudflare)	1 - 5 €
📦 Azure Blob Storage	5 - 15 €
🔒 Tailscale (VPN)	0 €
🔥 Monitoring (Prometheus + Grafana)	0 €
🔄 Reverse Proxy (HAProxy + Nginx)	0 €
🗄️ Base de données (PostgreSQL / Galera)	0 €
📦 Virtualisation (VirtualBox + Vagrant)	0 €
📁 Stockage distribué (GlusterFS)	0 €



Total estimé

~10 à 40 € / mois

2. INFRASTRUCTURE CLOUD COMPLÈTE



~700 à 1000 € / mois

Environnement cloud complet
Haute disponibilité et scalabilité

Élément	Coût estimé / mois
📦 Compute (VMs, App Services)	300 - 500 €
📁 Stockage (Blob / Disques)	100 - 200 €
🗄️ Base de données managée	100 - 200 €
🌐 Réseau (Load Balancer, VPN, DNS)	50 - 100 €
📊 Monitoring & Logs	50 - 100 €
🔄 Sauvegarde & Réplication	50 - 100 €
🛡️ Sécurité (WAF, Firewall, IAM)	50 - 100 €
... Divers / Imprévus	50 - 100 €



Total estimé

~700 à 1000 € / mois

3. INFRASTRUCTURE PHYSIQUE PROFESSIONNELLE



> 100 000 € CAPEX

Infrastructure on-premise
Investissement initial important

Composant	Estimation (CAPEX)
📦 Serveurs (haute performance)	40 000 - 60 000 €
📁 Stockage (NAS / SAN)	20 000 - 30 000 €
🌐 Réseau (Switchs, Routeurs, Pare-feu)	10 000 - 20 000 €
🏢 Infrastructure physique (rack, onduleur, clim.)	10 000 - 20 000 €
📄 Licences logicielles	10 000 - 20 000 €
🔧 Installation & Intégration	10 000 - 20 000 €
⚙️ Maintenance annuelle (en option)	10 000 - 20 000 €



Investissement initial

> 100 000 € CAPEX

+ coûts récurrents annuels



À RETENIR

- 🔍 Le POC hybride est idéal pour tester et valider à faible coût.
- 🌐 Le cloud complet offre performance, élasticité et haute disponibilité.
- 🏢 L'infrastructure physique garantit le contrôle total mais demande un investissement élevé.

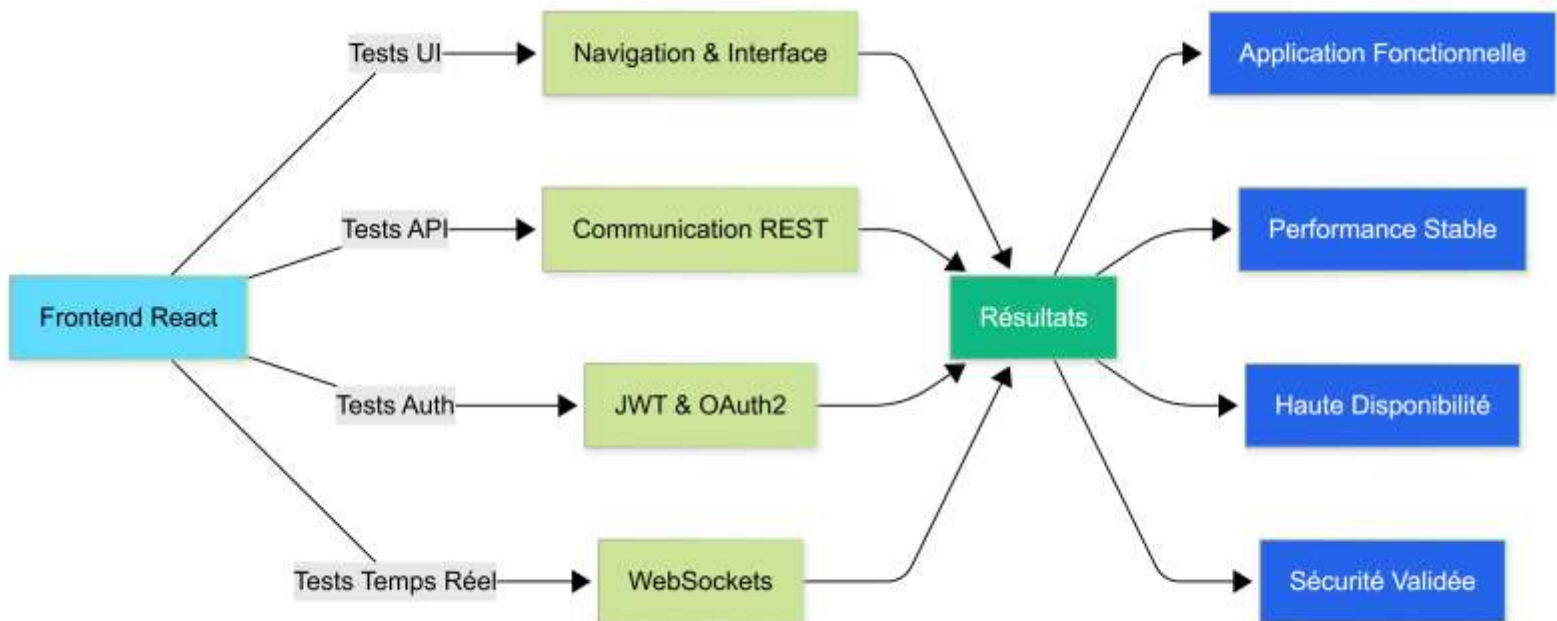


RECOMMANDATION

Commencer par l'infrastructure POC hybride
puis évoluer vers le cloud selon les besoins.

7 Tests et résultats :

Afin de valider le bon fonctionnement de l'architecture SUPFile, plusieurs séries de tests ont été réalisées sur les différentes couches de l'infrastructure : frontend, backend, réseau, sécurité et haute disponibilité. Ces tests avaient pour objectif de vérifier la stabilité des services, la communication entre les composants ainsi que la résilience globale de la plateforme.



7.1 Tests réalisés :

Plusieurs catégories de tests ont été mises en place afin de valider le fonctionnement complet de l'application.

Tests frontend :

Les tests frontend ont permis de vérifier :

- Le chargement correct des pages React ;
- La navigation entre les différentes routes ;
- L'affichage dynamique des composants ;
- La compatibilité responsive ;

- La stabilité de l'interface utilisateur.
- Des tests ont également été réalisés sur :
- Les formulaires ;
- Les uploads de fichiers ;
- Les notifications utilisateur ;
- La gestion des erreurs frontend.

Tests API :

Les endpoints REST FastAPI ont été testés à l'aide de :

- Swagger/OpenAPI ;
- Requêtes HTTP ;
- Outils de tests API.

Les tests ont permis de valider :

- Les réponses serveur ;
- La gestion des requêtes authentifiées ;
- Les échanges JSON ;
- Les uploads et téléchargements ;
- La stabilité des services backend.

Les communications entre Nginx, Gunicorn et FastAPI ont également été vérifiées.

Tests d'authentification :

Les systèmes d'authentification JWT et OAuth2 ont été testés afin de vérifier :

- La génération des tokens ;
- La validation des sessions ;
- Les expirations ;
- Les redirections OAuth ;
- La sécurisation des accès.

Les connexions via :

- Google ;
- GitHub ;
- Microsoft Azure ;

Ont été intégrées dans l'infrastructure frontend/backend

Tests de communication inter-services :

Des tests réseau ont été réalisés afin de valider :

- Les communications entre datacenters ;
- Les tunnels Tailscale/WireGuard ;
- Les communications API inter-services ;
- La réplication des données ;
- Le fonctionnement des reverse proxy.

Les mécanismes de haute disponibilité ont également été testés :

- Failover HAProxy ;
- Keepalived ;
- GlusterFS ;
- Galera Cluster.

Ces tests ont permis de vérifier la continuité de service lors de l'arrêt de certains nœuds.

7.2 Résultats obtenus :

Les différents tests réalisés ont permis de valider la stabilité globale de l'architecture SUPFile.

Fonctionnalités opérationnelles :

Les principales fonctionnalités de la plateforme sont opérationnelles :

- Authentification utilisateur ;
- upload de fichiers ;
- Téléchargement ;
- Partage ;
- Communication frontend/backend ;
- Gestion des sessions ;
- Monitoring ;
- Haute disponibilité.

L'architecture distribuée fonctionne correctement entre les différents datacenters du POC.

Performance :

Les performances obtenues sont satisfaisantes dans le cadre du projet :

- Chargement rapide du frontend React ;
- Bonnes performances API FastAPI ;
- Temps de réponse réduits grâce à Nginx et HAProxy ;
- Optimisation du build frontend avec Vite.

La répartition de charge permet également de mieux distribuer les requêtes entre les différents serveurs.

Disponibilité :

L'infrastructure assure une disponibilité élevée grâce :

- À l'architecture Active/Active ;
- Aux mécanismes de failover ;
- À la réplication des données ;
- Aux systèmes de backup.

Les tests de bascule ont montré que les services restent accessibles même après l'arrêt de certains serveurs.

Stabilité :

Les services backend et frontend ont montré une bonne stabilité après optimisation de :

- Gunicorn ;
- Nginx ;
- WebSockets ;
- Reverse proxy ;
- Configuration réseau.

Les problèmes initiaux de communication frontend/backend ont été résolus grâce à l'amélioration de la configuration réseau et proxy.

Sécurité :

Les mécanismes de sécurité mis en place permettent de protéger efficacement l'infrastructure :

- HTTPS ;
- JWT ;
- OAuth2 ;
- Tailscale/WireGuard ;
- Segmentation VLAN ;
- Reverse proxy sécurisé ;
- Contrôle des accès.

Les communications entre les datacenters restent chiffrées et sécurisées.

8 Conclusion :

Le projet SUPFile a permis de concevoir et de déployer une plateforme de stockage cloud distribuée reposant sur une architecture moderne, sécurisée et hautement disponible. L'intégration de technologies frontend récentes comme React, TypeScript et Vite, combinée à une infrastructure backend basée sur FastAPI, PostgreSQL, HAProxy et GlusterFS, a permis de construire une solution complète capable de répondre aux contraintes de sécurité, de résilience et de scalabilité d'une application cloud. Le projet a également permis de mettre en œuvre des concepts avancés liés à la virtualisation, au DevOps, à la haute disponibilité, aux VPN sécurisés inter-datacenter et à l'orchestration d'infrastructures hybrides. Les différents tests réalisés ont validé le bon fonctionnement global du système ainsi que la stabilité des communications entre les différents services. Enfin, ce projet a permis d'acquérir des compétences techniques importantes en développement frontend/backend, administration systèmes et réseaux, cybersécurité, cloud computing et architecture distribuée, tout en ouvrant des perspectives d'amélioration futures telles que la conteneurisation Kubernetes, l'automatisation CI/CD avancée, l'intégration d'outils d'observabilité supplémentaires et le déploiement sur une infrastructure cloud de production complète.